Angus Chan

Docker Tutorial Responses

What user are you logged in as by default?
The user is logged in as root.

If you start and then exit an interactive container, and then use the docker run -it ubuntu:xenial /bin/bash command again, is it the same container? How can you tell?
It is not the same container because the container ID number and the unique human-readable name are different. Use 'docker ps -a' to output the ID numbers.

```
anguschan@Anguss-MacBook-Pro-64 ~ % docker ps -a
CONTAINER ID   IMAGE           COMMAND       CREATED          STATUS                      PORTS     NAMES
2b69f440dce9   fedora:latest   "/bin/bash"   14 minutes ago   Exited (0) 7 minutes ago              quirky_tesla
21c0c84a7f02   ubuntu:xenial   "/bin/bash"   16 minutes ago   Exited (0) 14 minutes ago             funny_tesla
da6d0f627524   ubuntu          "bash"        24 minutes ago   Exited (0) 17 minutes ago             quirky_goldstine
13d50127c3ea   hello-world     "/hello"      26 minutes ago   Exited (0) 26 minutes ago             focused_haslett
```

Run the image you just built. Since we specified the default CMD, you can just do docker run -it mypython:latest. What do you observe?
Python 3.6.9 is running inside the container. GCC 8.4.0 on Linux means that Python is running on the Linux kernel. 'Type "help", "copyright", "credits" or "license" for more information' is the Python Read-Eval-Print-Loop (REPL) welcome notice. '>>>' is where the code is entered.

```
anguschan@Anguss-MacBook-Pro-64 docker_python % docker run -it mypython:latest
Python 3.6.9 (default, Mar 10 2023, 16:46:00)
[GCC 8.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Write and build a Dockerfile that installs the packages fortune and fortunes-min and runs the fortune executable (located in /usr/games/fortune after you install it). Note that you won't need to use the -it flags when you run the container as fortune doesn't need STDIN. Submit your Dockerfile with this lab. Hint: if you're having trouble writing your Dockerfile, try booting an interactive container and installing both packages. Translate what you did into a Dockerfile. How can you translate what you did interactively to a Dockerfile?

Paste the output of your docker images command after questions 1 and 2.
Question 1

```
anguschan@Anguss-MacBook-Pro-64 docker_python % docker run -it mypython:latest
Python 3.6.9 (default, Mar 10 2023, 16:46:00)
[GCC 8.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Question 2

```
anguschan@Anguss-MacBook-Pro-64 docker_python % docker run my-fortune-app
Whoever has lived long enough to find out what life is, knows how deep a debt
of gratitude we owe to Adam, the first great benefactor of our race.  He
brought death into the world.
                -- Mark Twain, "Pudd'nhead Wilson's Calendar"
```

With httpd running in a detached container, run /bin/bash on the same container and paste the output of ps aux. Observe that there's very few processes running as compared to running ps aux on your VM. Why is this the case?

```
root@ceee113c5534:/usr/local/apache2# ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.1   8672  4688 ?        Ss   13:59   0:00 httpd -DFOREGROUND
www-data       8  0.0  0.1 1935948 4552 ?        Sl   13:59   0:00 httpd -DFOREGROUND
www-data       9  0.0  0.1 2001484 4560 ?        Sl   13:59   0:00 httpd -DFOREGROUND
www-data      10  0.0  0.1 2001484 4560 ?        Sl   13:59   0:00 httpd -DFOREGROUND
root          98  0.0  0.0   4176  3484 pts/0    Ss   14:04   0:00 /bin/bash
root         227  100  0.0   6016  3268 pts/0    R+   14:05   0:00 ps aux
```

A container does not run a full operating system, as it shares the host's kernel and runs the httpd process. Virtual machines, on the other hand, boots its own kernels and runs a full operating system; hence ps aux shows far more processes and their process IDs are much larger.

Paste your Dockerfile for the Node.js web application

Pushed to the repository.

Paste your docker-compose.yml file

Pushed to the repository.